

# Documento Técnico

## WebSec Analyzer — Asistente Inteligente para Análisis de Seguridad Web

**Materia:** Herramientas de Ciberseguridad  
**Profesor:** Pablo Náchez  
**Institución:** Universidad Iberoamericana León  
**Fecha:** Mayo 2026

### 1. Definición del Problema

Las revisiones de seguridad web generan resultados técnicos que, en muchos casos, resultan difíciles de interpretar para personas sin experiencia en ciberseguridad. Un administrador de sistemas puede ejecutar una herramienta como Nikto o OWASP ZAP y obtener decenas de líneas de salida técnica sin saber cuál es prioritaria, qué impacto real tiene cada hallazgo o qué acción concreta debe tomar.

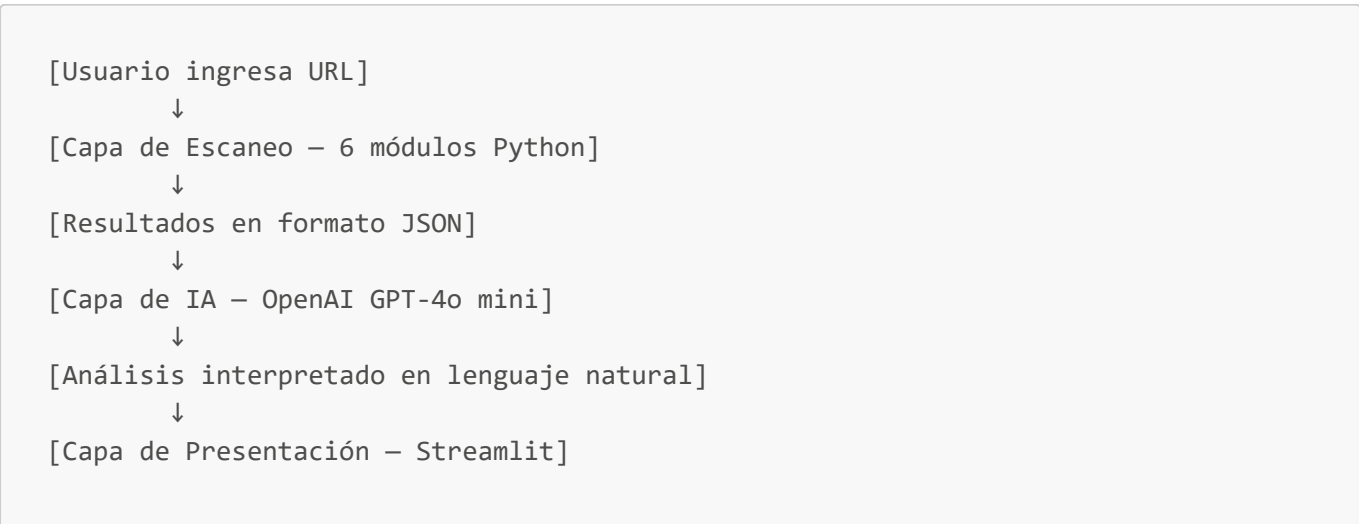
Este proyecto busca resolver ese problema construyendo una herramienta que automatice un análisis básico de seguridad web de forma no destructiva y, mediante inteligencia artificial, convierta los hallazgos técnicos en información comprensible y accionable para cualquier tipo de usuario.

### 2. Objetivos

- 1. Analizar un sitio web autorizado ejecutando validaciones básicas no destructivas.
- 2. Identificar configuraciones, exposiciones o debilidades visibles que representen riesgo para la información.
- 3. Integrar un componente de IA que interprete los hallazgos técnicos en lenguaje comprensible.
- 4. Permitir la interacción con el sistema mediante un asistente conversacional.
- 5. Generar resultados útiles tanto para usuarios técnicos como para usuarios no técnicos.

### 3. Arquitectura del Sistema

El sistema está construido sobre tres capas principales que interactúan de forma secuencial:



↓

[Interfaz web + Bot conversacional]

### 3.1 Estructura de Archivos

```
proyecto-seguridad/
├── app.py                # Aplicación principal y orquestador
├── requirements.txt      # Dependencias del proyecto
├── .env                 # Variables de entorno (API key)
├── scanner/
│   ├── headers.py       # Módulo: encabezados HTTP de seguridad
│   ├── ssl_check.py     # Módulo: validación SSL/TLS
│   ├── tech_detect.py   # Módulo: detección de tecnologías
│   ├── ports.py         # Módulo: escaneo de puertos
│   ├── forms.py         # Módulo: detección de formularios
│   └── exposure.py      # Módulo: rutas y archivos expuestos
└── ai/
    └── analyst.py       # Integración con OpenAI: análisis y bot
```

## 4. Módulos Técnicos — Validaciones de Seguridad

### 4.1 Encabezados HTTP de Seguridad (headers.py)

**Descripción:** Este módulo realiza una petición HTTP GET al sitio objetivo mediante la librería `requests` y extrae los encabezados de respuesta. Los compara contra una lista de 7 encabezados de seguridad que toda aplicación web debería implementar según las mejores prácticas de OWASP.

**Encabezados evaluados:**

| Encabezado                | Propósito                                                          |
|---------------------------|--------------------------------------------------------------------|
| Strict-Transport-Security | Fuerza el uso de HTTPS, previene ataques MITM                      |
| Content-Security-Policy   | Controla los recursos que puede cargar el sitio, previene XSS      |
| X-Frame-Options           | Previene clickjacking bloqueando el uso del sitio en iframes       |
| X-Content-Type-Options    | Evita que el navegador interprete archivos con tipo incorrecto     |
| Referrer-Policy           | Controla qué información de referencia se comparte entre sitios    |
| Permissions-Policy        | Restringe el acceso a APIs del navegador (cámara, micrófono, etc.) |
| X-XSS-Protection          | Activa el filtro XSS nativo del navegador (soporte legacy)         |

**Salida:** Lista de encabezados presentes y ausentes, con su descripción y valor actual si están configurados.

### 4.2 Validación SSL/TLS (ssl\_check.py)

**Descripción:** Establece una conexión directa al puerto 443 del servidor usando los módulos estándar `ssl` y `socket` de Python. Extrae y valida la información del certificado digital del sitio.

**Datos recolectados:**

- Uso de HTTPS en la URL ingresada
- Validez del certificado (cadena de confianza)
- Fecha de expiración y días restantes
- Versión del protocolo TLS negociada (TLS 1.0, 1.1, 1.2 o 1.3)
- Entidad emisora del certificado (CA)
- Nombre del sujeto (Common Name)

**Criterios de riesgo:** Un certificado con menos de 30 días de vigencia, el uso de TLS 1.1 o inferior, o un certificado auto-firmado sin emisor reconocido representan riesgos de seguridad.

---

### 4.3 Detección de Tecnologías (`tech_detect.py`)

**Descripción:** Descarga el HTML de la página y el conjunto completo de encabezados HTTP, luego realiza una búsqueda de patrones o "firmas" asociadas a tecnologías web conocidas. Este enfoque es similar al que utilizan herramientas como WhatWeb o Wappalyzer.

**Tecnologías detectables:** WordPress, Joomla, Drupal, Shopify, Wix, React, Vue.js, jQuery, Bootstrap, nginx, Apache, Cloudflare, PHP, ASP.NET, entre otras.

**Método:** Coincidencia de cadenas de texto (substrings) entre el contenido del sitio y un diccionario de firmas predefinidas. Por ejemplo, si el HTML contiene `wp-content`, el sistema infiere que el sitio usa WordPress; si el encabezado `Server` contiene `Apache/2.4`, se registra el servidor web y su versión exacta.

**Riesgo asociado:** Revelar la versión exacta del servidor (ejemplo: `Apache/2.4.66`) permite a un atacante buscar vulnerabilidades conocidas (CVEs) específicas para esa versión.

---

### 4.4 Escaneo de Puertos (`ports.py`)

**Descripción:** Realiza un escaneo básico de conectividad TCP sobre un conjunto de puertos comunes, intentando establecer una conexión con `socket.connect_ex()` y un timeout de 2 segundos por puerto. No es un escaneo agresivo ni utiliza técnicas de evasión — únicamente verifica si el puerto responde.

**Puertos evaluados:**

| Puerto | Servicio |
|--------|----------|
| 21     | FTP      |
| 22     | SSH      |
| 23     | Telnet   |
| 25     | SMTP     |
| 80     | HTTP     |

| Puerto | Servicio                |
|--------|-------------------------|
| 443    | HTTPS                   |
| 3306   | MySQL                   |
| 3389   | RDP (Escritorio Remoto) |
| 8080   | HTTP alternativo        |
| 8443   | HTTPS alternativo       |

**Interpretación:** Puertos como el 22 (SSH), 3306 (MySQL) o 3389 (RDP) expuestos públicamente representan superficies de ataque que deberían estar protegidas con reglas de firewall.

---

## 4.5 Detección de Formularios ([forms.py](#))

**Descripción:** Descarga el HTML de la página y lo parsea con [BeautifulSoup](#) buscando etiquetas `<form>`, `<input>`, `<textarea>` y `<select>`. Para cada formulario encontrado extrae su método HTTP, la URL de destino (acción) y la lista de campos que contiene.

### Aspectos evaluados:

- Número total de formularios en la página
- Método de envío (GET o POST) — los formularios con datos sensibles deben usar POST
- Presencia de campos de tipo `password`
- URL de destino del formulario

**Relevancia de seguridad:** Un formulario de login que use método GET enviaría las credenciales como parte de la URL, exponiéndolas en logs del servidor, historial del navegador y proxies intermedios.

---

## 4.6 Exposición de Rutas y Archivos ([exposure.py](#))

**Descripción:** Prueba un conjunto de rutas sensibles predefinidas haciendo peticiones HTTP individuales y registrando el código de respuesta. Identifica recursos que deberían estar protegidos o no existir en producción.

**Rutas evaluadas:** `/robots.txt`, `/.env`, `/admin`, `/administrator`, `/login`, `/wp-admin`, `/backup`, `/config`, `/phpinfo.php`, `/server-status`, `/.git/HEAD`, `/api`, `/swagger`, `/api-docs`, `/.htaccess`, `/web.config`

### Criterio de riesgo:

- HTTP 200 → Riesgo Alto (el archivo existe y es accesible)
  - HTTP 301/302 → Riesgo Medio (redirección, puede revelar estructura)
  - HTTP 403 → Riesgo Medio (existe pero está protegido)
  - HTTP 404 → Sin riesgo detectado
- 

# 5. Módulo de Inteligencia Artificial ([analyst.py](#))

## 5.1 Modelo Utilizado

Se integra la API de OpenAI utilizando el modelo **GPT-4o mini**, seleccionado por su balance entre capacidad de razonamiento, velocidad de respuesta y costo. El modelo recibe los resultados del scanner serializado en formato JSON y genera interpretaciones estructuradas en español.

## 5.2 Funciones de IA Implementadas

El sistema cumple con las 5 funciones mínimas requeridas:

| Función                       | Implementación                                                                   |
|-------------------------------|----------------------------------------------------------------------------------|
| Resumir hallazgos             | Resumen ejecutivo en 2-3 oraciones con lenguaje accesible                        |
| Clasificar / priorizar riesgo | Tabla de hallazgos con nivel Alto / Medio / Bajo                                 |
| Explicar impacto              | Descripción del impacto real de los 2 hallazgos más críticos                     |
| Sugerir mitigaciones          | Top 3 de acciones concretas y accionables                                        |
| Lenguaje ejecutivo            | Todo el análisis está redactado para ser comprensible sin conocimientos técnicos |

## 5.3 Funcionamiento del Bot Conversacional

El asistente mantiene un historial de mensajes por sesión. Cada vez que el usuario hace una pregunta, se envía a la API el contexto completo: el prompt del sistema (que incluye los resultados del scanner), el historial de conversación previo y la nueva pregunta. Esto permite que el bot responda con coherencia y sin perder el contexto de análisis previo.

## 5.4 Consideraciones sobre Alucinaciones

Dado que el modelo recibe datos estructurados reales como input y no se le pide que genere información sobre el sitio de forma autónoma, el riesgo de alucinación es bajo. El modelo actúa como intérprete de los datos reales — no como generador de hallazgos. Sin embargo, las interpretaciones de impacto y mitigación son generadas por el modelo y deben tratarse como orientativas, no como auditoría de seguridad certificada.

---

## 6. Interfaz de Usuario (**app.py**)

La interfaz está construida con **Streamlit**, un framework de Python que permite crear aplicaciones web interactivas sin necesidad de HTML, CSS o JavaScript. La interfaz se divide en dos columnas:

- **Columna izquierda:** Campo de ingreso de URL, botón de análisis, indicadores de progreso por módulo, análisis de la IA y resultados detallados organizados en secciones expandibles con código de colores por nivel de riesgo.
- **Columna derecha:** Chat conversacional con historial de mensajes donde el usuario puede hacer preguntas sobre los hallazgos.

---

## 7. Capacidad de Usuarios Simultáneos

Streamlit maneja cada sesión de usuario de forma independiente mediante el objeto `st.session_state`. En un servidor local (como el utilizado para este prototipo), la capacidad práctica es de **5 a 10 usuarios simultáneos** sin degradación notable del rendimiento, limitada principalmente por:

- El ancho de banda disponible para las peticiones del scanner
- El tiempo de respuesta de la API de OpenAI (promedio 3-8 segundos)
- Los recursos de CPU y RAM del equipo local

Para un despliegue en producción con mayor concurrencia, se recomendaría usar **Streamlit Community Cloud** o un servidor dedicado con al menos 2 vCPUs y 4 GB de RAM, lo que elevaría la capacidad a 50-100 sesiones concurrentes.

---

## 8. URL Pública y Despliegue

Al ejecutar la aplicación con el comando `streamlit run app.py`, Streamlit genera dos URLs de acceso:

```
Local URL:    http://localhost:8501
Network URL:  http://[IP-local]:8501
```

La **Network URL** permite acceso desde cualquier dispositivo conectado a la misma red WiFi, lo que es suficiente para una demostración en el salón de clases. Para exponer la aplicación a internet de forma temporal (sin configurar un servidor), se puede usar **ngrok**:

```
ngrok http 8501
```

Esto genera una URL pública temporal del tipo `https://xxxx.ngrok.io` accesible desde cualquier navegador con conexión a internet.

Para un despliegue permanente y gratuito, **Streamlit Community Cloud** ([streamlit.io/cloud](https://streamlit.io/cloud)) permite publicar la aplicación directamente desde un repositorio de GitHub con un dominio del tipo `https://nombre-app.streamlit.app`.

---

## 9. Stack Tecnológico

| Componente      | Tecnología         | Versión       |
|-----------------|--------------------|---------------|
| Lenguaje        | Python             | 3.10+         |
| Interfaz web    | Streamlit          | Latest        |
| Peticiones HTTP | requests           | Latest        |
| Parsing HTML    | BeautifulSoup4     | Latest        |
| SSL/TLS         | ssl, socket        | Stdlib Python |
| IA              | OpenAI GPT-4o mini | API v1        |

| Componente           | Tecnología    | Versión |
|----------------------|---------------|---------|
| Variables de entorno | python-dotenv | Latest  |

## 10. Limitaciones del Sistema

1. **No es una auditoría certificada.** La herramienta realiza validaciones básicas y no reemplaza una auditoría de seguridad profesional (pentesting, VAPT).
2. **El escaneo de puertos es superficial.** Solo evalúa 10 puertos comunes con timeout de 2 segundos. Herramientas como Nmap realizan escaneos mucho más exhaustivos.
3. **La detección de tecnologías es por firmas predefinidas.** Si el sitio oculta sus tecnologías o usa una menos común, no será detectada.
4. **No analiza el interior de la aplicación.** No realiza pruebas autenticadas, no evalúa lógica de negocio, ni detecta vulnerabilidades en el código fuente.
5. **Dependencia de la API de OpenAI.** Si la API no está disponible o la key ha alcanzado su límite, el módulo de IA no funcionará.
6. **URL local por defecto.** Sin configuración adicional, la app solo es accesible en la red local del equipo donde corre.

## 11. Consideraciones Éticas y Legales

- El sistema fue diseñado exclusivamente para análisis **no destructivo** de sitios web **autorizados**.
- No realiza explotación de vulnerabilidades, fuerza bruta, evasión de controles ni denegación de servicio.
- No almacena ni transmite información privada de terceros.
- Los resultados generados son orientativos y deben usarse con criterio responsable.
- El uso de esta herramienta sobre sitios sin autorización expresa puede constituir un delito informático conforme a la legislación mexicana (Código Penal Federal, artículos 211 bis 1 al 211 bis 7).
- Todo análisis realizado durante el desarrollo de este proyecto fue efectuado sobre entornos académicos y sitios propios.

## 12. Conclusión

WebSec Analyzer demuestra que es posible combinar herramientas de código abierto, inteligencia artificial y una interfaz accesible para construir un sistema de análisis de seguridad útil tanto para perfiles técnicos como no técnicos. La integración de GPT-4o mini como capa interpretativa convierte resultados técnicos en recomendaciones comprensibles, cerrando la brecha entre los hallazgos de seguridad y la toma de decisiones informada.